dMIR Peephole Rewrites for Consensus-Critical EVM JITs with First-Class Carry Operators

Anonymous Author(s)

Anonymous Institution
anonymous@example.com

Abstract. Peephole rewrites in blockchain JITs must preserve transaction semantics bit-for-bit; divergence between consensus-path VMs can cause chain forks. Lowering U256 arithmetic to 64-bit hardware creates multi-word carry chains, but LLVM/Alive2 overflow intrinsics expose carry through multi-result values, splitting carry-dependent rewrites across SSA nodes. This blocks many rules from single-query equivalence checks and leaves engineers to drop them, hand-prove them, or ship them unverified. dMIR (DTVM's middle IR) represents carry and overflow as first-class bit-vector operators, so each dMIR rewrite becomes a Z3 equivalence query within a two-tier optimization architecture. An offline Z3 admission and coverage pipeline checks all 70 production dMIR rewrites under fixed-width bit-vector semantics, including carry-chain rewrites enabled by ADC/SBB operators. A separate x86 CgIR layer adds 13 cleanup rewrites validated by matched differential testing rather than SMT. The full 83-rule system achieves a 7.24% geometric-mean speedup on 27 evmone-bench workloads, with all 5,884 Cancun differential tests reporting byte-identical final state and matching gas accounting across the suite (correctness is bounded by suite coverage). This design addresses a verification bottleneck for consensus-critical EVM JITs with a domain-specific IR whose carry and overflow semantics are SMT-checkable.

Keywords: Blockchain information systems · EVM JIT · dMIR · Peephole optimization · SMT equivalence checking · Carry chain · Translation validation

1 Introduction

The execution layer of a blockchain information system is a replicated consensus state machine: every node on the consensus path must produce bit-identical results for the same transaction. The EVM is a widely deployed smart-contract execution environment; when a client enables a JIT, the compiler becomes part of the consensus-critical path. Peephole rewrites in the JIT act directly during machine-code generation, so their correctness directly affects the determinism of on-chain execution; an incorrect equivalence judgment, deployed on only some clients, may trigger a chain fork.

Existing formal verification of EVM peephole rewrites mostly targets the block-level bytecode abstraction. The machine-level patterns emitted by a JIT during code generation—especially the multi-word carry chains that arise when the 256-bit word size is lowered onto 64-bit hardware—sit at a different abstraction layer from bytecode rules and are awkward to validate as standard LLVM/Alive2 peephole queries. The root cause is how IRs model bit-level side results. In LLVM IR, basic arithmetic is defined only on N-bit integers; the carry bit must be retrieved through an intrinsic that returns a multi-return tuple of {result, carry}. Rules depending on this carry bit can be expressed only through indirect structures in Alive [11] and similar tools; the bottleneck is the IR's expressiveness, not the SMT solver [12]. For a consensus-critical JIT, an unchecked rule class leaves the engineer with three choices—drop it (performance), ship it without verification (safety), or hand-prove it (scalability).

Our approach is to extend the IR with first-class bit-vector operators so that SMT-checkable peephole rules cover a larger rule class, without modifying the solver or relaxing the differential-test threshold. In dMIR, the bit-level patterns that previously required indirect multi-return encoding are represented directly, so the corresponding rules fall within Z3 bit-vector equivalence. The SMT scope is dMIR arithmetic/flag semantics; gas, memory, storage, and x86 equivalence remain outside it.

Concretely, we make three contributions:

- **Carry-aware dMIR semantics.** We promote carry and overflow from implicit side results to first-class dMIR operators, so ADC/SBB-style multi-word dependencies become single bit-vector equivalence obligations rather than indirect multi-return encodings.
- **SMT admission and production coverage.** An offline Z3 pipeline admits all **70** production dMIR rewrites under fixed-width modular bit-vector semantics. Its enumeration also reaches all production left-hand sides (**70/70**), including carry-chain forms that resist direct formulation in LLVM IR.
- **Layered optimization with consensus-path evidence.** SMT-checked dMIR arithmetic simplifications compose with x86-64 CgIR cleanups validated by matched differential testing; across 27 evmone-bench workloads, the 83-rule system attains a **+7.24%** geometric-mean speedup and passes **5,884** Cancun differential tests with byte-identical final state and matching gas accounting within suite coverage.

2 Related Work

2.1 Peephole Optimization Frameworks

We position our work by abstraction layer, verification method, and carry-chain coverage. Peephole synthesis and verification frameworks provide the closest methodological baseline, but they do not by themselves provide a complete solution for EVM JIT-backend carry chains. Classical synthesis-then-verification systems include Denali [9], Bansal–Aiken [4], STOKE [18], Stratified Synthesis [7], and egg [19]; SMT-based peephole checking is exemplified by Alive [11], Alive2 [10], Souper [16], and PEEK [15]; dataflow filtering [13] accelerates search; recent LLM-driven peephole synthesis on LLVM IR [20] is a generative complement to deductive enumeration. We follow the verify-after-enumeration paradigm but move the checked IR from LLVM IR to dMIR, addressing the multi-return carry and overflow encoding limit that motivates this paper.

LeanMLIR [5] formalizes IR-parametric peephole verification in Lean 4, whereas this paper reports a concrete EVM JIT rule set. Hydra [14] generalizes bit-width-independent rules but, like Alive2, builds on LLVM IR and inherits the multi-return tuple limit on the four carry-chain examples in Table 1; closing that gap via an auxiliary lifted encoding is orthogonal to our IR-extension contribution. Schett and Nagele [17] enumerate SMT-checked EVM *bytecode* rules, a higher abstraction layer than the JIT code-generation path we target with dMIR matching and x86 cleanup.

2.2 Verification Work on the Consensus-Critical Path

Albert et al. [2] give a Coq proof of AOT EVM bytecode-block rules. Our scope sits at lower abstraction layers: JIT IR and machine-code, SMT admission at synthesis time, and multi-word carry chains. Seven rules overlap: 7/83 in our set and 7/14 in their `forves` set. This suggests complementary scope rather than a superset relationship: the remaining 76 cover dMIR algebraic and Boolean simplifications (including ADC/SBB carry-chain rewrites that motivate the first-class carry operators) plus 13 x86 cleanup rewrites, none of which are reachable from the bytecode-block abstraction.

Runtime divergence detection—Hydra Framework [6], multi-client detectors, EVM fuzzing, consensus test networks—catches faults after deployment; our workflow validates rules before deployment. Bytecode-level EVM optimization [3,8,1] is orthogonal to our JIT IR/code-generation focus. Existing work therefore does not jointly address JIT-backend rewrites, multi-word carry chains, automated SMT admission, and consensus-path determinism; this combination is the gap targeted by our dMIR-based design.

3 SMT-Checked dMIR Peephole Rewrites

The EVM word size is 256 bits, while commodity registers are 64 bits. After JIT code generation, a U256 addition becomes one 64-bit `add` followed by three carry-propagating `adc` instructions; subtraction similarly uses one `sub` and three `sbb`. Figure 1 shows the dMIR and x86-64 forms for a representative 4-limb addition and its serial carry chain.

Among all instructions emitted by the JIT, instructions directly tied to U256 multi-word arithmetic account for only **2.15%** (weighted mean across the U256-heavy subset of our 27 benchmarks—the U256-heavy cluster in Figure 2). The chain is nonetheless worth optimizing: every `adc`/`sbb` on the chain depends on the carry output of the previous instruction, forming a strict *serial data dependency* that defeats instruction-level parallelism.

dMIR	*1:1 lowering*	**x86-64**
`r0 = add(a0, b0)`		`add rax, rcx`
	$\downarrow c_0$	
r1 = ADC(a1, b1, c_0)		`adc rbx, rdx`
	$\downarrow c_1$	
r2 = ADC(a2, b2, c_1)		`adc rsi, rdi`
	$\downarrow c_2$	
r3 = ADC(a3, b3, c_2)		`adc r8,  r9`

Fig. 1. U256 4-limb addition in dMIR (left) and x86-64 (right). Each `ADC` lowers 1:1 to `adc`; dashed arrows trace the serial carry chain c_0, c_1, c_2 that the lowering preserves.

On U256-heavy workloads, rule hit rates reach 25–99.9% with a very uneven distribution; on non-U256-heavy benchmarks (sha1 family, `jump_around`) the U256 rule hit rate is well below that range, and gains come from x86-layer mechanical cleanup.

Hardware cost is only one side of the picture. The other side is whether existing peephole-verification frameworks can reason about such carry chains at all.

In LLVM IR, basic arithmetic (`add`, `sub`, `mul`) is defined only on N-bit integers `iN`; carry comes from `@llvm.uadd.with.overflow.iN`, a `{iN, i1}` tuple rather than a first-class arithmetic operator. Following the Alive/Alive2 [11,10] framework, Table 1 lists four carry-chain rules that Alive2 cannot directly verify (tuple type mismatch, or timeout in our lifted encoding). The obstacle is carry modeling rather than bit-vector reasoning itself.

Table 1. Four representative carry-chain rules that Alive2 (based on LLVM IR) cannot directly verify for equivalence because of the multi-return tuple encoding. The first three are single `ADC`/`SBB` rules where the right-hand side drops the carry output; the fourth is a 4-limb carry-chain merge rule.

Rule	Rewrite
`adc_zero_carry`	`ADC`$(x, y, 0) \to$ `add`(x, y)
`sbb_zero_borrow`	`SBB`$(x, y, 0) \to$ `sub`(x, y)
`sbb_self_zero`	`SBB`$(x, x, 0) \to 0$
4-limb ADC chain	$4 \times$ `uadd.w.overflow` $\to$ `add.i256`

The four rules' Alive2 command lines, failure messages, and tool versions (Alive2 v21.0, Z3 4.8.12) are archived in the artifact.

By contrast, six U256 rules without multi-return carry pass Alive2 in ~60 ms each. In LLVM IR, expressing one carry-propagating addition takes five lines of scaffolding around `uadd.with.overflow`; in dMIR a single `ADC` suffices, bringing all four rules into the scope of automatic equivalence checking. The JIT runs a compiled two-tier implementation: dMIR for algebraic rewrites and x86 CgIR (code-generation IR) for carry-chain cleanup.

3.1 dMIR Bit-Vector Semantics and Carry-Chain Operators

The core data type of dMIR is a parametric-width bit-vector BV_n with BV_1 representing carry-in and carry-out bits (c_{in}, c_{out}). The carry-propagating addition has signature `ADC` : $\mathrm{BV}_n \times \mathrm{BV}_n \times \mathrm{BV}_1 \rightarrow \mathrm{BV}_n \times \mathrm{BV}_1$; `SBB` has analogous borrow-propagation semantics. In Z3 [12], carry propagation zero-extends both 256-bit operands and the 1-bit carry to 257 bits, sums once, and slices: the low 256 bits are the result and bit 256 is carry out. This embeds carry propagation entirely in bit-vector arithmetic with no multi-return tuple or auxiliary predicate. The model covers dMIR arithmetic and flags; gas, memory aliasing, storage, and x86 equivalence remain outside it. The same semantics covers ordinary operators (`add`, `sub`, `mul`, `and`, `or`, `xor`, shifts); the carry-chain operator is the operator that general-purpose IRs lack.

Trusted Base. The SMT check assumes the dMIR bit-vector model agrees with the JIT runtime; the 5,884 differential tests provide end-to-end enabled-vs-baseline regression evidence. Fuzzing on randomly generated bytecode remains future work.

3.2 Candidate Enumeration and SMT Admission

Admission Check. For a candidate $LHS \rightarrow RHS$, we submit to Z3 the proposition $\exists \boldsymbol{v}.\ LHS(\boldsymbol{v}) \neq RHS(\boldsymbol{v})$. *Unsat* admits the rule under fixed-width modular bit-vector semantics; *sat* or timeout rejects (sat returns a counterexample). Equivalence assumes nothing beyond modular arithmetic—no signed interpretation and no undefined-overflow assumptions. All 70 production dMIR rules pass the offline Z3 admission check, and CI re-runs this check on every change to the rule file.

Two Enumeration Runs. We report two separate enumeration experiments with disjoint goals. The *capacity run* fixes a single grammar—a depth-2 algebraic enumerator plus a small set of hand-written carry templates—and measures the Z3 admission rate; this characterizes solver throughput and where false positives tend to arise. The *coverage run* extends the same algebraic enumerator with additional left-hand-side operator tops and a constant-pool widening, plus a pattern-template instantiation channel reusing the template miner configuration; this measures whether the tuned offline search space can reach the 70 hand-designed dMIR production rules, not blind recall on an unseen rule set.

The capacity run uses two complementary paths: *(i) depth-2 algebraic enumeration*—all left/right combinations of depth ≤ 2 over the dMIR arithmetic and Boolean operators, covering common algebraic identities; *(ii) carry-template enumeration*—multi-word carry-chain candidates constructed by hand around the carry-chain operators.

Throughput and Admission Rate. To measure how much of the capacity-run search space survives SMT admission, and where rejected candidates concentrate, we group generated candidates by enumeration path and outcome; Section 4 reports the full table. The capacity run admits 765/775 algebraic candidates (98.7%, 10 timeouts) and 17/78 carry templates (21.8%, 61 semantic rejections): carry-template admission is limited by 61 semantic rejections, mostly width-mismatch edge cases (full breakdown in the artifact), while the algebraic side has only 10 timeouts and zero semantic rejections. These candidates measure enumeration capacity; only the curated production rules are loaded into the JIT at runtime.

Production Rule Coverage (Methodology). The coverage run asks whether the search space contains the 70 hand-designed production dMIR rules. We compare canonicalized left-hand sides: alpha-normalization renames variables to $x_1, x_2, \ldots$ in traversal order, and commutative pairs (e.g. $\texttt{or}(x, y)$ and $\texttt{or}(y, x)$) collapse to one representative. A production rule is *covered* when some enumerated rule has the same canonical left-hand side; the *false-miss rate* is the fraction with no such match. The enumerated right-hand side must be Z3-equivalent, but need not syntactically match the production right-hand side.

Shadow Audit and Recursive Commutative-Aware Matching. The shadow audit runs alongside production matching in two modes: a baseline that swaps only at the rule's root, and a commutative-aware mode that tries swaps recursively at every commutative-rooted subtree to depth four over $\{\text{add}, \text{and}, \text{or}, \text{xor}, \text{mul}\}$. The production codegen is extended in lockstep so emitted matchers mirror the same recursive policy. On the existing 70 production rules the extension is byte-identical—no commutative-rooted rule has a nested commutative subtree—so it ships as production-ready infrastructure for any future rule with a nested commutative left-hand side.

Corpus-Driven Methodology Check. A separate enumerator pass rebuilds the rule corpus from a canonicalized opcode-bigram model of a Cancun trace; we use it as a methodology cross-check on the corpus enumeration pipeline itself, distinct from the hand-driven curation that selects the 70 production rules. The corpus pass reproduces, byte-identical to the prior corpus audit, the same audit-accepted subset of 1,432 rules; equivalence between the two paths is the contribution, not a yield gain. The audit-accepted denominator used below is 1,432.

Counterexamples. A rejected carry template illustrates why SMT admission is essential on the chain: $\texttt{SBB}(x, x, c_f) \to 0$ fails at $x{=}0, c_f{=}1$ (LHS is $\texttt{0xFF}\ldots\texttt{F}$; correct form $\texttt{sub}(0, c_f)$), reflecting a modular-carry trap on ignored carry input. The artifact includes the enumerator, template miner, canonical comparator, Z3 admission checks, rule-set JSON, coverage output, and unit tests.

Before evaluating performance, we separate the rule set by abstraction layer and evidence type so that SMT-checked dMIR rewrites are not conflated with differentially tested x86 cleanup rules. Table 2 gives this classification.

Table 2. Classification of the two-tier peephole rewrite rules by evidence type. The dMIR rules are checked for equivalence by Z3 on the bit-vector semantic model; the x86 CgIR rules are covered by matched differential tests, not by SMT.

Layer	Evidence	Category	Count
dMIR	Z3 equivalence check	Boolean normalization	42
		Algebraic identity	15
		Identity elimination	10
		Constant folding	2
		Dead-code elimination	1
		Subtotal	**70**
x86 CgIR	Differential tests	Redundant compare/test cleanup	8
		Branch-fallthrough cleanup	2
		No-op elimination	2
		test-setcc merge	1
		Subtotal	**13**
Total			**83**

Two-Tier Rule Set. The dMIR layer handles machine-independent algebraic simplification (all 70 dMIR rules pass Z3); the x86 CgIR layer handles hardware-specific cleanup on registers, flags, and branches (13 syntactic rewrites with **no separate SMT script**, covered by the matched differential suites summarized in Table 4); a formal x86 SMT model remains future work.

Boolean normalization and algebraic identities together cover most dMIR rules—these are precisely the patterns the canonical-form enumerator is designed to cover. Identity-elimination and shift-zero patterns admit Z3-discharged proofs in milliseconds, while the carry-chain class is the one that motivates the operator extension introduced earlier in this section.

Selection Asymmetry. The 70 dMIR production rules are selected manually from recurring profile hotspots and lowering patterns, then submitted one by one to Z3; this is hotspot-guided curation, not a claim of systematic optimality. The enumeration pipeline is a measurement channel: it does not drive production rule maintenance, and the 1966 admitted candidates are not automatically promoted to production. The x86-layer rules are covered by code review and matched test suites.

4 Evaluation

4.1 Enumeration and Admission Output vs. the General-Purpose IR

The capacity run asks how many offline candidates survive Z3 admission in the dMIR search space, and whether the same U256 sample can be checked in a general-purpose LLVM-IR verifier. Table 3 summarizes the admission output; the Alive2 run below isolates the remaining expressiveness gap.

Table 3. Capacity-run enumeration/admission output; the two candidate paths are described in §3.2.

Candidate source	Input	Pass	SAT cex	Timeout	Admission rate
Algebraic enumeration	775	765	0	10	98.7%
Carry templates	78	17	61	0	21.8%

SAT counterexamples are semantic rejections; timeouts are rejected conservatively.

On a 10-rule U256 capacity-run sample, Alive2 verifies the 6 without multi-return carry but reports type mismatch or timeout on the 4 `adc`/`sbb` cases. This sample is separate from the four production-style cases in Table 1; both expose the same tuple-encoding limitation.

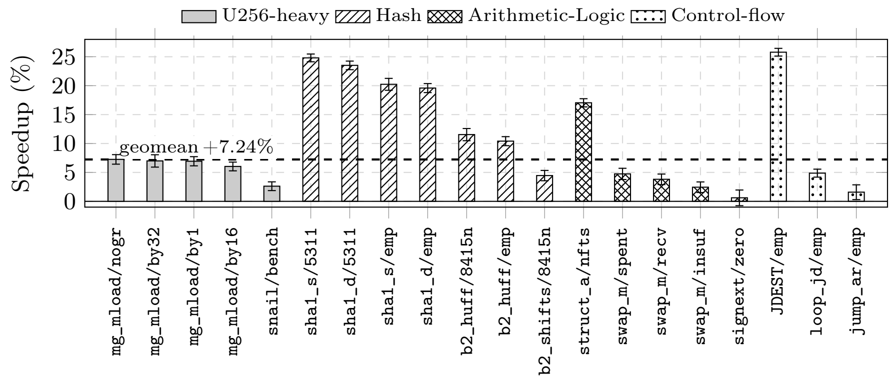
Production Rule Coverage. Using the canonical-LHS methodology, the coverage run produces **1966** Z3-admitted rewrite rules: **1856** from the extended algebraic enumerator and **110** from pattern-template instantiation. The pattern-template path starts from 2964 candidates, of which 110 survive Z3 verification and deduplication against the enumerator output and the existing rule file. On the 70 hand-designed production rules, the run matches **69/69** unique canonical patterns—one commutative-twin pair, `or`(x, y) versus `or`(y, x), shares a single canonical representative that both production names match—equivalently **70/70** production rule names; the false-miss rate against the production set is therefore **0/70**.

Scope of the Coverage Claim. The enumerator's left-hand-side operator tops and constant pool were tuned in light of an earlier 0/70 baseline that exposed mechanical gaps (missing tops such as `not`, `select`, `shl`, `ushr`, `sshr`; missing small constants; structural-equality dedup of `and`(x, x)). Reaching 70/70 therefore shows the production rules lie inside an enumerable Z3-discharged search space, not blind recall of an unseen target. Coverage is on canonical left-hand sides only—enumerated right-hand sides are Z3-equivalent to their own left-hand sides but are not syntactically required to match the production rule's right-hand side. The tuned search space is broad enough to contain every hand-designed rule, with 1966 members passing Z3 admission; non-production members are not claimed to be useful runtime rewrites.

4.2 End-to-End Performance

After admission, the next question is whether the production rules produce end-to-end speedups on real EVM workloads rather than merely enlarging the pool of locally valid rewrites. We compare the rules-enabled build with upstream main on evmone-bench; positive point-estimate speedup cases appear in Figure 2.

Setup. Intel Core Ultra 7 265K/Ubuntu 22.04; evmone-bench, baseline (upstream `main`) vs. rules-enabled build, 20 runs (30 if CV $\geq$ 3%), bootstrap CIs (B=10 000), pinned CPU, turbo off.



Labels: mg=`memory_grow`, b2=`blake2b`, sha1_s=`sha1_shifts`, sha1_d=`sha1_divs`, struct_a=`struct_array`, swap_m=`swap_method`, signext=`signextend`, JDEST=`JUMPDEST_n0`, loop_jd=`loop_jumpdest`, jump_ar=`jump_around`, snail=`snailtracer`.

Fig. 2. Positive point-estimate speedup configurations over upstream main. Error bars are 95% bootstrap CIs (B=10 000); the dashed line marks the **+7.24%** 27-benchmark geometric mean. Bars are grouped as U256-heavy, Hash, Arithmetic-Logic, and Control-flow.

Across all 27 benchmarks, the full 83-rule set improves the geometric mean by **+7.24%** (95% CI [+2.59%, +12.11%]). Among the positive cases, peak speedup is **+25.79%** on `JUMPDEST_n0/empty`. The `sha1` family gains +19.59%–+24.81%, and `blake2b_huff` gains +10.41%–+11.54%.
Hit rate and speedup are decoupled: `mg_mload` matches nearly every U256 instruction yet gains under 8%, while sha1 hits only 7.5% of opcodes yet gains over 20% from x86 branch and compare-merge cleanup; blake2b follows the same pattern. The layers complement each other: multi-word rules concentrate gains on U256-heavy contracts, while CgIR handles broader hardware-specific cleanups outside machine-independent equivalence proofs.

Compilation Overhead. Across 776 JIT unit-test modules, median per-module compile time is 0.45 ms (p95 0.87 ms) for the full pipeline—an upper bound, not the incremental rewrite-pass cost (per-pass attribution is future work).

4.3 Shadow Audit and Statistical Validation

Noise Floor Calibration. We calibrate a paired benchmark noise floor in two stages: 0A measures within-window pairwise deltas (one prod-only build, 30 reps × 27 benchmarks × 4 windows; $p_{99} = \mathbf{1.86}\%$), 0B measures inter-run deltas under the same single-binary, environment-variable swap protocol used by the Stage 4 single-rule benchmark ($p_{99} = \mathbf{2.65}\%$). With a 1.5× inflation guard against carry-over correlation, the effective threshold for declaring a per-rule speedup is **2.80%**; the pilot fixes 15 reps per candidate at this threshold (α=0.05, power 0.80, median CV 0.73%).

Dual-Arm Audit on 1,432 Rules. We run the matcher in two configurations on the audit-accepted subset of 1,432 rules: a *naive* arm with root-only swap, and an *aware* arm with recursive commutative-aware swap to depth four. Only **1** rule (`synth-add-1907`, the identity `add`$(x, 0)$) shows a non-zero recovery delta of **48** additional fires across the trace—about **0.02%** of the 217,325 total audit hits across the 1,432 rules (or 0.06% of `synth-add-1907`'s own 76,842 hits). The remaining 1,431 rules are cold in the benchmark trace under both arms.

Single-Rule Bench and FDR. The four highest-firing candidates plus three pre-registered negative controls (`null_dead_code`, `null_depth5`, `null_self_cancel`) are benchmark-validated with 15 reps × 27 benchmarks × 2 sides per candidate, fresh process per run, bracketed-mean delta, bootstrap CI (B=10 000). Across the resulting 189 (rule, bench) hypotheses, the gate $\Delta<-2.80\% \wedge \text{ci\_high}<0 \wedge q_{\text{BH}}<0.05$ is satisfied by **0** pairs, and the negative-control gate also holds (0 of 3 valid): the pipeline records $n_{\text{picks}} = 0$. The contribution is the methodology—turning matcher-capability claims into a strict statistical test—rather than the rule yield.

4.4 Correctness and Rule Coverage

The final question is whether enabling the 83-rule set changes observable EVM behavior relative to the baseline. We therefore run matched differential suites and compare final state and gas accounting between the two builds; the pass counts are summarized in Table 4.

Table 4. Differential test results for the rules-enabled build (83 added rules) and upstream-main baseline: passes/total.

Test dimension	Enabled	Baseline	Suite source
Unit (JIT)	223/223	223/223	EVM spec subset
Unit (interpreter)	215/215	215/215	Run list
statetest (JIT)	2723/2723	2723/2723	Cancun 101 suites
statetest (interpreter)	2723/2723	2723/2723	Cancun 101 suites
Total	**5884/5884**	**5884/5884**	—

Both sides agree item by item across the four differential categories (5884/5884 total). Independently, `evmone-statetest` on Cancun confirms the same pass matrix under multipass JIT and interpreter modes (Cancun selected via `evmone-statetest -k fork_Cancun`; Prague excluded because DTVM does not yet support that fork). The suite covers Cancun state tests and EVM unit tests; opcode- and gas-branch coverage completeness are not claimed. CI re-runs the same admission check on every rule-file change.

Codegen-Extension Regression. The recursive commutative-aware codegen extension passes the four suites of Table 4. The lone in-tree `erc20 ctest` failure is pre-existing: the baseline reproduces it, and regenerated production-rule artifacts are byte-identical pre/post patch.

5 Conclusion

This paper extends dMIR with first-class carry and overflow operators and builds an SMT-checked peephole layer on top: 70 dMIR rules pass Z3 admission over arithmetic and flags, and 13 x86 cleanup rules are covered by matched differential testing. The combination delivers a $+7.24\%$ geometric-mean speedup on evmone-bench while preserving byte-identical final state across 5,884 Cancun differential tests, with correctness bounded by suite coverage.

The shadow audit adds a stricter measurement path: a recursive matcher over commutative subtrees plus bench-paired CI-bounded validation reports no statistically valid recovery on the current production set ($n_{\text{picks}} = 0$), so the result is methodological and negative rather than a rule yield. Future work is to formalize the x86 CgIR tier, check cross-implementation equivalence across EVM engines such as evmone and revm, and generate carry-chain optimizations automatically through equality saturation or superoptimization.

Data and Code Availability

Rule sets, Z3 admission checks, evaluation scripts, and data are available from an anonymous repository upon request to the program chairs and will be de-anonymized upon acceptance. CI runs the production dMIR validation tests on every rule-file change.

References

1. Aguiar, M.A., Albert, E., Genaim, S., Gordillo, P., Hernández-Cerezo, A., Kirchner, D., Rubio, A.: Neural-guided superoptimization in Ethereum. Inf. Softw. Technol. (2025). doi:10.1016/j.infsof.2025.107800
2. Albert, E., Genaim, S., Kirchner, D., Martin-Martin, E.: Formally verified EVM block-optimizations. In: Proc. CAV. LNCS, vol. 13966, pp. 176–189 (2023). doi:10.1007/978-3-031-37709-9_9
3. Albert, E., Gordillo, P., Rubio, A., Schett, M.A.: Synthesis of super-optimized smart contracts using Max-SMT. In: Proc. CAV. LNCS (2020). doi:10.1007/978-3-030-53288-8_10

4. Bansal, S., Aiken, A.: Automatic generation of peephole superoptimizers. In: Proc. ASPLOS (2006). doi:10.1145/1168857.1168906
5. Bhat, S., Keizer, A., Hughes, C., Goens, A., Grosser, T.: Verifying peephole rewriting in SSA compiler IRs. In: Proc. ITP. LIPIcs, vol. 309, pp. 9:1–9:20 (2024). doi:10.4230/LIPIcs.ITP.2024.9
6. Breidenbach, L., Daian, P., Tramèr, F., Juels, A.: Hydra: Principled bug bounties for smart contract security. In: Proc. USENIX Security (2018)
7. Heule, S., Schkufza, E., Sharma, R., Aiken, A.: Stratified synthesis: Automatically learning the x86-64 instruction set. In: Proc. PLDI (2016). doi:10.1145/2908080.2908121
8. Hu, X., Zhao, D., Scholz, B.: Synthesizing efficient super-instruction sets for Ethereum virtual machine. In: Proc. VMIL (2024). doi:10.1145/3689490.3690403
9. Joshi, R., Nelson, G., Randall, K.: Denali: A goal-directed superoptimizer. In: Proc. PLDI (2002). doi:10.1145/512529.512566
10. Lopes, N.P., Lee, J., Hur, C.K., Liu, Z., Regehr, J.: Alive2: Bounded translation validation for LLVM. In: Proc. PLDI (2021). doi:10.1145/3453483.3454030
11. Lopes, N.P., Menendez, D., Nagarakatte, S., Regehr, J.: Provably correct peephole optimizations with alive. In: Proc. PLDI (2015). doi:10.1145/2737924.2737965
12. de Moura, L., Bjørner, N.: Z3: An efficient SMT solver. In: Proc. TACAS (2008). doi:10.1007/978-3-540-78800-3_24
13. Mukherjee, M., Kant, P., Liu, Z., Regehr, J.: Dataflow-based pruning for speeding up superoptimization. Proc. ACM Program. Lang. **4**(OOPSLA) (2020). doi:10.1145/3428245
14. Mukherjee, M., Regehr, J.: Hydra: Generalizing peephole optimizations with program synthesis. Proc. ACM Program. Lang. **8**(OOPSLA1) (2024). doi:10.1145/3649837
15. Mullen, E., Zuniga, D., Tatlock, Z., Grossman, D.: Verified peephole optimizations for CompCert. In: Proc. PLDI (2016). doi:10.1145/2908080.2908109
16. Sasnauskas, R., Chen, Y., Collingbourne, P., Ketema, J., Taneja, J., Regehr, J.: Souper: A synthesizing superoptimizer (2017)
17. Schett, M.A., Nagele, J.: Populating the peephole optimizer of a smart contract compiler. In: Proc. FMBC. OASIcs (2020). doi:10.4230/OASIcs.FMBC.2020.3
18. Schkufza, E., Sharma, R., Aiken, A.: Stochastic superoptimization. In: Proc. ASPLOS (2013). doi:10.1145/2451116.2451150
19. Willsey, M., Nandi, C., Wang, Y.R., Flatt, O., Tatlock, Z., Panchekha, P.: egg: Fast and extensible equality saturation. Proc. ACM Program. Lang. **5**(POPL) (2021). doi:10.1145/3434304
20. Xu, Z., Xu, H., Tian, Y., Zhou, X., Sun, C.: LPO: Discovering missed peephole optimizations with large language models. In: Proc. ASPLOS (2026). doi:10.1145/3779212.3790184